1. Identify missing values — isnull().sum()
Use: Identifying missing values helps us understand whether the dataset is complete and reliable. In our sales dataset, columns such as Quantity, UnitPrice, OrderDate, and City are important for analysing sales. If values are missing, calculations and reports may become inaccurate. The isnull().sum() command counts missing values in every column, helping us identify exactly where the problem exists.

Possible causes:

- The employee forgot to enter the quantity or price when recording an order.
- Some order details were not provided or captured.
- Data was imported incorrectly from an Excel file or another system.
- Invalid values, such as "invalid-date", were converted into missing dates using errors="coerce".
- Numeric values such as "invalid" were converted into missing values using pd.to_numeric() with errors="coerce".

Example: If the Quantity column has two missing values, we cannot calculate the exact revenue for those orders until we find or estimate the missing quantities.
Missing values can be handled using fillna() to replace them with suitable values or dropna() to remove affected rows. In our dataset, we filled missing quantities with 0, missing prices with the median, and missing cities with "Unknown". We also removed the row with an invalid order date.

2. Identify duplicate orders — duplicated()
Use: Identifying duplicates helps us find repeated records in a dataset. In our sales dataset, the same order may accidentally be entered more than once. Duplicate records can increase the calculated number of orders and inflate total revenue, making the business appear to have more sales than it actually did.
df.duplicated().sum()
To display the duplicate rows:
df[df.duplicated()]
Possible causes:

- An employee entered the same order twice.
- The same file was imported into the system more than once.
- Data was copied and pasted incorrectly.
- An automated sales system sent the same record multiple times.
- Multiple items legitimately belong to the same order ID, so a repeated ID does not always mean a duplicate.

Example: In our original dataset, there were two duplicate records involving Order IDs 1002 and 1008.
We removed exact duplicate rows using:
df = df.drop_duplicates()
After removing them, df.duplicated().sum() returns 0.
To identify repeated Order IDs specifically, we can use:
df[df.duplicated("OrderID", keep=False)]

This displays all rows with repeated Order IDs, including the first occurrence. We should inspect these records before deleting them because two different products can legitimately share the same order ID.

3. Find the highest unit price — .max()

Use: Finding the maximum unit price helps us identify the most expensive product in the dataset. A business can use this information to understand its premium products, review pricing, and compare product prices. The .max() method returns the highest available numeric value in a column.

Expected result for our dataset:
500000.0
To retrieve the complete record with the highest unit price:
df.loc[df["UnitPrice"].idxmax()]
This returns the details of Order ID 1016, which has a unit price of ₹500,000.

Possible causes:

- The product is genuinely expensive.
- An employee accidentally added an extra zero while entering the price.
- A currency or decimal-formatting mistake occurred during data entry.
- An incorrect value was imported from another system.
- A high-priced product is a genuine outlier compared with the other products.

Example: If the highest unit price is ₹500,000 while most products cost between ₹500 and ₹60,000, it may be a data-entry mistake or an unusual transaction. However, we should verify the original order before changing the value.
Important distinction:

- .max() identifies the highest unit price.
- .idxmax() identifies the index of the row containing that price.
- Using .loc[] with .idxmax() retrieves the complete order record.

The highest unit price is not necessarily the order with the highest revenue, because revenue depends on both quantity and unit price.

4. Group revenue by city — groupby()

Use: Grouping revenue by city helps us understand which locations generate the most sales. This information supports business decisions such as allocating stock, planning deliveries, running promotions, and deciding where to expand operations. By grouping the data by City and adding the Revenue values, we can compare total revenue across locations.

df.groupby("City")["Revenue"].sum()

Possible causes:

- One city has more customers than another.
- Some locations receive larger orders or sell more products.

- Certain products are more popular in particular cities.
- Prices, discounts, or seasonal demand vary by location.
- Missing quantities or prices can make revenue calculations incomplete.
- Inconsistent city names, such as "Chennai", "chennai", and " Chennai ", can split the same location into separate groups if text is not standardised.
- Duplicate orders inflate the recorded revenue for a city.

Example: If Chennai generates ₹5,000 in revenue and Madurai generates ₹3,000, Chennai has higher recorded revenue in the dataset. The business can investigate whether this is due to more orders, larger quantities, higher-priced products, or greater customer demand.

In our dataset, we standardised city names using:

```
df["City"] = df["City"].str.strip().str.title()
```

We also created the Revenue column using:

```
df["Revenue"] = df["Quantity"] * df["UnitPrice"]
```

This allows us to calculate total revenue for each city consistently.